# Preliminary System Design Proposal

**COMP5355 Cyber and Internet Security (2025/2026)**
**Project:** End-to-End Encrypted Communication System

---

## 1. Chosen Task

**Task 1: End-to-End Encrypted Messaging System**

Our group will implement a one-to-one E2EE text messaging system. Plaintext exists only on sender and recipient devices; a relay server handles registration, public-key distribution, and ciphertext routing but never decrypts message content.

---

## 2. System Design Sketch

### 2.1 Architecture

The system consists of three components:

| Component | Role |
|---|---|
| **Sender / Recipient (CLI clients)** | Generate and store private keys locally; perform authenticated key exchange; encrypt and decrypt messages |
| **Relay Server (FastAPI + SQLite)** | User registration, password-based login, public-key storage, and transparent relay of handshake and message envelopes |
| **Network (untrusted)** | Carries all traffic; provides no confidentiality, integrity, or authenticity |

**Implementation stack:** Python 3.11+, PyNaCl / `cryptography`, FastAPI, HTTP polling CLI clients.

### 2.2 Key Management

Each user generates locally:

- **Ed25519 identity signing key (IK_sig)** — authenticates the sender during session setup
- **X25519 identity DH key (IK_dh)** — static Diffie–Hellman contribution
- **X25519 signed prekey (SPK)** — published to the server for session initiation

The server stores only public key material. Private keys never leave the client.

### 2.3 Protocol Overview

**Registration:** Client generates key pairs, uploads a signed key bundle (`IK_sig`, `IK_dh`, `SPK`, signature), and registers with a username and password.

**Session establishment (X3DH-lite):** Before the first message, the sender:

1. Fetches the recipient's public key bundle from the server
2. Generates a fresh **ephemeral X25519 key (EK)**
3. Computes a shared secret from multiple DH exchanges (EK↔recipient keys, sender IK↔recipient SPK)
4. Derives a **session key (SK)** via HKDF-SHA256
5. Signs the handshake transcript with **IK_sig**; the recipient verifies before accepting

6. Destroys ephemeral private key material after SK derivation (**forward secrecy**, Bonus B1)

**Messaging:** Each message is encrypted with **XChaCha20-Poly1305 (AEAD)**. Associated data binds `session_id`, sender, recipient, direction, and a monotonic **sequence number**. The server relays opaque ciphertext envelopes without parsing plaintext.

**Malicious-server mitigation (Bonus B2):** Clients display a **safety number** (SHA-256 fingerprint of the peer's identity public key) for out-of-band verification before trusting a key bundle.

## 2.3 Message Flow (High Level)

```
[Alice CLI] --register/login--> [Relay Server] <--register/login-- [Bob CLI]
[Alice CLI] --fetch Bob's key bundle--> [Relay Server]
[Alice CLI] --handshake_init (signed, ephemeral PK)--> [Relay Server] --> [Bob CLI]
[Bob CLI] --handshake_resp--> [Relay Server] --> [Alice CLI]
[Alice CLI] --encrypted message (AEAD)--> [Relay Server] --> [Bob CLI]
```

Plaintext is visible only inside the two CLI processes. The server sees ciphertext, routing metadata, and public keys only.

## 2.4 Scope

**In scope:** User registration, authenticated key exchange, online one-to-one text messaging, SR1–SR4, and optional Bonus extensions (forward secrecy, malicious-server resistance).

**Out of scope:** Group messaging, offline store-and-forward, attachments, metadata hiding, and availability under sustained drop/delay attacks.

---

# 3. Intended Threat Model

We adopt the project's mandatory adversaries and extend them for bonus goals.

## 3.1 Trust Boundaries

| Entity | Assumption |
|---|---|
| **Endpoints (Alice & Bob)** | Trusted to run the protocol correctly and protect long-term and session secrets while in use |
| **Relay Server (base model)** | **Honest-but-curious (A3):** follows the protocol but may read and log all relayed data; must not learn plaintext |
| **Network** | Fully adversarial: no built-in confidentiality, integrity, or authenticity |
| **Cryptographic primitives** | Standard primitives from vetted libraries (PyNaCl) are secure |

**Explicit assumption:** Client-to-server login may optionally use TLS to protect passwords; **E2EE security does not rely on TLS**. Message confidentiality and integrity are established entirely at the application layer between endpoints.

## 3.2 Adversary Classes

| ID | Adversary | Capabilities |
|---|---|---|
| **A1** | Passive network attacker | Observes all traffic (ciphertext, sizes, timing, addresses) |
| **A2** | Active network attacker | A1 + modify, drop, delay, reorder, replay, or inject messages; may attempt MITM during setup |
| **A3** | Honest-but-curious server | Reads all stored/relayed data; does not actively tamper |
| **A4** *(Bonus)* | Transient endpoint compromise | One-time read of a device's secret state (e.g., lost phone); no persistent control |
| **A5** *(Bonus)* | Malicious server | A3 + may tamper with relayed data or distribute fake public keys |

## 3.3 Security Requirements

| ID | Requirement | Planned Mechanism |
|---|---|---|
| **SR1** | Confidentiality | AEAD encryption; SK known only to endpoints |
| **SR2** | Integrity | AEAD authentication tag; reject invalid ciphertext |
| **SR3** | Sender authenticity | Signed handshake (Ed25519) + SK derived from authenticated exchange |
| **SR4** | Replay protection | Monotonic per-session sequence numbers; reject duplicates |
| **SR5** *(Bonus)* | Forward secrecy | Ephemeral DH in every session; delete EK and SK after use |
| **SR6** *(Bonus)* | Malicious-server resistance | AEAD + seq binding; out-of-band safety-number verification of peer identity keys |

## 3.4 Allowed Leakage

We explicitly allow leakage of ciphertext length, message routing metadata (sender, recipient, timestamps), and public key material on the server. Plaintext and session keys remain endpoint-only.

---

*Group members: [Name 1], [Name 2], [Name 3] — please fill in before submission.*